

CA3075: Insecure DTD Processing (code analysis)

CA3075: Insecure DTD Processing (code analysis) - gewarren, learn.microsoft.com.

- Published: date not stated
- Original: <https://learn.microsoft.com/en-us/dotnet/fundamentals/code-analysis/quality-rules/ca3075>
- Preserved from: <https://learn.microsoft.com/en-us/dotnet/fundamentals/code-analysis/quality-rules/ca3075> (preserved-copy) on 2026-08-04
- Licence: CC BY 4.0

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

CA3075: Insecure DTD Processing

Summarize this article for me

Property	Value	Rule ID	CA3075	Title	Insecure DTD Processing	Category	Security	Fix is breaking or non-breaking	Non-breaking	Enabled by default in .NET 10	No	Applicable languages	C# and Visual Basic
----------	-------	---------	--------	-------	-------------------------	----------	----------	---------------------------------	--------------	-------------------------------	----	----------------------	---------------------

Cause

If you use insecure [DtdProcessing](#) instances or reference external entity sources, the parser may accept untrusted input and disclose sensitive information to attackers.

Rule description

A *Document Type Definition (DTD)* is one of two ways an XML parser can determine the validity of a document, as defined by the [World Wide Web Consortium \(W3C\) Extensible Markup Language \(XML\) 1.0](#). This rule seeks properties and instances where untrusted data is accepted to warn developers about potential [Information Disclosure](#) threats or [Denial of Service \(DoS\)](#) attacks. This rule triggers when:

-

DtdProcessing is enabled on the [XmlReader](#) instance, which resolves external XML entities using [XmlUrlResolver](#).

-

The [InnerXml](#) property in the XML is set.

-

[DtdProcessing](#) property is set to Parse.

-

Untrusted input is processed using [XmlResolver](#) instead of [XmlSecureResolver](#).

-

The [XmlReader.Create](#) method is invoked with an insecure [XmlReaderSettings](#) instance or no instance at all.

-

[XmlReader](#) is created with insecure default settings or values.

In each of these cases, the outcome is the same: the contents from either the file system or network shares from the machine where the XML is processed will be exposed to the attacker, or DTD processing can be used as a DoS vector.

How to fix violations

-

Catch and process all [XmlTextReader](#) exceptions properly to avoid path information disclosure.

-

Use the [XmlSecureResolver](#) to restrict the resources that the [XmlTextReader](#) can access.

-

Do not allow the [XmlReader](#) to open any external resources by setting the [XmlResolver](#) property to **null**.

-

Ensure that the [DataViewManager.DataViewSettingCollectionString](#) property is assigned from a trusted source.

.NET Framework 3.5 and earlier

-

Disable DTD processing if you are dealing with untrusted sources by setting the [ProhibitDtd](#) property to **true**.

-

[XmlTextReader](#) class has a full trust inheritance demand.

.NET Framework 4 and later

-

Avoid enabling DtdProcessing if you're dealing with untrusted sources by setting the [XmlReaderSettings.DtdProcessing](#) property to **Prohibit** or **Ignore**.

-

Ensure that the Load() method takes an XmlReader instance in all InnerXml cases.

Note

This rule might report false positives on some valid XmlSecureResolver instances.

When to suppress warnings

Unless you're sure that the input is known to be from a trusted source, do not suppress a rule from this warning.

Suppress a warning

If you just want to suppress a single violation, add preprocessor directives to your source file to disable and then re-enable the rule.

```
#pragma warning disable CA3075
// The code that's violating the rule is on this line.
#pragma warning restore CA3075
```

To disable the rule for a file, folder, or project, set its severity to `none` in the [configuration file](#).

```
[*.{cs,vb}]
dotnet_diagnostic.CA3075.severity = none
```

For more information, see [How to suppress code analysis warnings](#).

Pseudo-code examples

Violation 1

```

using System.IO;
using System.Xml.Schema;

class TestClass
{
    public XmlSchema Test
    {
        get
        {
            var src = "";
            TextReader tr = new StreamReader(src);
            XmlSchema schema = XmlSchema.Read(tr, null); // warn
            return schema;
        }
    }
}

```

Solution 1

```

using System.IO;
using System.Xml;
using System.Xml.Schema;

class TestClass
{
    public XmlSchema Test
    {
        get
        {
            var src = "";
            TextReader tr = new StreamReader(src);
            XmlReader reader = XmlReader.Create(tr, new XmlReaderSettings() { XmlResolver
= null });
            XmlSchema schema = XmlSchema.Read(reader, null);
            return schema;
        }
    }
}

```

Violation 2

```

using System.Xml;

namespace TestNamespace
{
    public class TestClass
    {
        public XmlReaderSettings settings = new XmlReaderSettings();
        public void TestMethod(string path)
        {
            var reader = XmlReader.Create(path, settings); // warn
        }
    }
}

```

Solution 2

```

using System.Xml;

namespace TestNamespace
{
    public class TestClass
    {
        public XmlReaderSettings settings = new XmlReaderSettings()
        {
            DtdProcessing = DtdProcessing.Prohibit
        };

        public void TestMethod(string path)
        {
            var reader = XmlReader.Create(path, settings);
        }
    }
}

```

Violation 3

```

using System.Xml;

namespace TestNamespace
{
    public class DoNotUseSetInnerXml
    {
        public void TestMethod(string xml)
        {
            XmlDocument doc = new XmlDocument() { XmlResolver = null };
            doc.InnerXml = xml; // warn
        }
    }
}

```

```

using System.Xml;

namespace TestNamespace
{
    public class DoNotUseLoadXml
    {
        public void TestMethod(string xml)
        {
            XmlDocument doc = new XmlDocument(){ XmlResolver = null };
            doc.LoadXml(xml); // warn
        }
    }
}

```

Solution 3

```

using System.Xml;

public static void TestMethod(string xml)
{
    XmlDocument doc = new XmlDocument() { XmlResolver = null };
    System.IO.StringReader sreader = new System.IO.StringReader(xml);
    XmlReader reader = XmlReader.Create(sreader, new XmlReaderSettings() { XmlResolver =
    null });
    doc.Load(reader);
}

```

Violation 4

```

using System.IO;
using System.Xml;
using System.Xml.Serialization;

namespace TestNamespace
{
    public class UseXmlReaderForDeserialize
    {
        public void TestMethod(Stream stream)
        {
            XmlSerializer serializer = new
XmlSerializer(typeof(UseXmlReaderForDeserialize));
            serializer.Deserialize(stream); // warn
        }
    }
}

```

Solution 4

```

using System.IO;
using System.Xml;
using System.Xml.Serialization;

namespace TestNamespace
{
    public class UseXmlReaderForDeserialize
    {
        public void TestMethod(Stream stream)
        {
            XmlSerializer serializer = new
XmlSerializer(typeof(UseXmlReaderForDeserialize));
            XmlReader reader = XmlReader.Create(stream, new XmlReaderSettings() {
XmlResolver = null });
            serializer.Deserialize(reader );
        }
    }
}

```

Violation 5

```

using System.Xml;
using System.Xml.XPath;

namespace TestNamespace
{
    public class UseXmlReaderForXPathDocument
    {
        public void TestMethod(string path)
        {
            XPathDocument doc = new XPathDocument(path); // warn
        }
    }
}

```

Solution 5

```

using System.Xml;
using System.Xml.XPath;

namespace TestNamespace
{
    public class UseXmlReaderForXPathDocument
    {
        public void TestMethod(string path)
        {
            XmlReader reader = XmlReader.Create(path, new XmlReaderSettings() {
XmlResolver = null });
            XPathDocument doc = new XPathDocument(reader);
        }
    }
}

```

Violation 6

```

using System.Xml;

namespace TestNamespace
{
    class TestClass
    {
        public XmlDocument doc = new XmlDocument() { XmlResolver = new XmlUrlResolver()
    };
    }
}

```

Solution 6

```

using System.Xml;

namespace TestNamespace
{
    class TestClass
    {
        public XmlDocument doc = new XmlDocument() { XmlResolver = null }; // or set to a
        XmlSecureResolver instance
    }
}

```

Violation 7

```

using System.Xml;

namespace TestNamespace
{
    class TestClass
    {
        private static void TestMethod()
        {
            var reader = XmlTextReader.Create("doc.xml"); //warn
        }
    }
}

```

```

using System.Xml;

namespace TestNamespace
{
    public class TestClass
    {
        public void TestMethod(string path)
        {
            try {
                XmlTextReader reader = new XmlTextReader(path); // warn
            }
            catch { throw ; }
            finally {}
        }
    }
}

```

Solution 7

```

using System.Xml;

namespace TestNamespace
{
    public class TestClass
    {
        public void TestMethod(string path)
        {
            XmlReaderSettings settings = new XmlReaderSettings() { XmlResolver = null };
            XmlReader reader = XmlReader.Create(path, settings);
        }
    }
}

```

Note

Although [XmlReader.Create](#) is the recommended way to create an [XmlReader](#) instance, there are behavior differences from [XmlTextReader](#). An [XmlReader](#) from [Create](#) normalizes `\r\n` to `\n` in XML values, while [XmlTextReader](#) preserves the `\r\n` sequence.